

High-Performance Vector Database: System Architecture & Benchmark Report

The vector database is separated into three components: the Vector Store, the Indexing Engine, and the Query Processor.

1.1 Vector Store

The Vector Store is the data layer. It handles raw vector ingestion and manages flat, contiguous memory buffers for optimal CPU cache locality. Manages append and update only vector buffers, generating unique vector identifiers, maintaining a map between global IDs and raw memory offsets, and making snapshots.

1.2 Indexing Engine

The Indexing Engine is responsible for optimizations for searching a vector. It transforms vectors into an IVF index structure. It executes clustering algorithms like K-Means and K-Means++, assigning vectors to their nearest cluster centroids.

1.3 Query Processor

The Query Processor executes all incoming search operations by translating user requests into optimized vector scans. It Parses query parameters like metric type, target, nprobe count, and mode.

2. Data Layout for Vector Store & IVF Index

To maximize hardware utilization, memory layouts are designed to avoid dynamic memory allocation and minimize cache misses during costly dot product and Euclidean distance loops.

2.1 Vector Store Core Memory Layout

Vectors are kept in a single, contiguously allocated block of memory representing using a `vector<float>`. A hashmap is used to map vector ids to their relative offset in the flat vector array.

2.2 Inverted File (IVF) Index Structure

The IVF Index breaks the continuous vector space into localized partitions. Structurally, it consists of two coordinated arrays one to store centroids and a 2d array to store all the vector ids belonging to the i th centroid.

3. K-Means Clustering & Centroid Assignment

3.1 The Cluster Count Rule

The target cluster count is dynamically computed using the underroot of the total ingested vectors.

3.2 Centroid Initialization: Random vs. K-Means++ (KPP)

There are two initialization modes:

KPP_OFF (Random Selection): K distinct vectors are picked at random from the dataset to serve as initial centroids. While fast, it frequently places multiple initial centroids within dense data regions, leaving sparse areas poorly covered and leading to more refinement loops.

KPP_ON (K-Means++): The first centroid is chosen at random. Each subsequent centroid is selected probabilistically from the remaining vectors with a probability proportional to its squared distance to the nearest already-chosen centroid.

3.3 Convergence Loop & Spherical Adjustments

Once initialized, the engine runs standard Lloyd's iteration loops:

1. Assignment: Each vector in the dataset is assigned to its nearest centroid using the specified metric (Euclidean or Cosine).
2. Update: Centroid coordinates are recomputed as the mean of all vectors assigned to them.
3. Termination: The loop breaks when no vectors change cluster assignments, or when the max iteration threshold is met.

4. Snapshot Serialization Format

The image is created using a temporary-file-and-rename strategy for crash safety purposes; first, the data is written to a temporary .tmp file and then renamed to the target filename atomically, preventing any corruption caused by crashes occurring while writing to the file.

The entire integer portion of the file will be in little-endian ordering. All floating point types shall utilize the IEEE 754 binary32 single precision type.

The data consists of a header with dimensions, number of vectors, and number of clusters (zero if no IVF index has been built). After the header, the file contains arrays of 64-bit IDs, the actual vectors, coordinates of centroids (if the IVF index has been created), and then for each cluster, the 32-bit array size followed by array entries with indices of vectors belonging to that particular cluster in 32-bit format.

During loading, the server reads the header, allocates the necessary arrays, loads the content, and rebuilds its internal hash map that associates external IDs with the internal positions. The server will automatically try to load the snapshot file on startup if such a file exists.

5. Concurrency Approach

In the server, the one-thread-per-client paradigm is used, meaning that each client connection works in its thread, thus enabling simultaneous processing of several clients.

There is a global mutex `mtx`, which protects the data from other threads working at the same time. This mutex must be locked whenever we access the data store (any operation such as ADD, SEARCH, BUILD, SAVE, or LOAD).

The client connections are handled by the `Threader` class, which keeps track of currently running client threads, waiting for the destructor of this class to join them.

6. Benchmark Analysis & Trade-Offs

6.1 Benchmark Results Table

=====							
=====							
METRIC	KPP	MODE	NPROBE	ITERATIONS	AVG TIME (ms)	AVG	
RECALL@10	VECS	SCANNED					
EUCLIDEAN	N/A	BRUTE	N/A	N/A	6.195	1.000	50000.0
EUCLIDEAN	KPP_OFF	IVF_STANDARD	1	83	0.094	0.067	224.4
EUCLIDEAN	KPP_OFF	IVF_STANDARD	5	83	0.346	0.202	1124.6
EUCLIDEAN	KPP_OFF	IVF_STANDARD	10	83	0.596	0.315	2249.3
EUCLIDEAN	KPP_OFF	IVF_STANDARD	25	83	1.393	0.537	5627.1
EUCLIDEAN	KPP_OFF	IVF_REFINED	1	83	4.111	0.794	16774.1
EUCLIDEAN	KPP_OFF	IVF_REFINED	5	83	4.163	0.794	16774.1
EUCLIDEAN	KPP_OFF	IVF_REFINED	10	83	4.116	0.794	16778.6
EUCLIDEAN	KPP_OFF	IVF_REFINED	25	83	4.162	0.797	16897.8
EUCLIDEAN	KPP_ON	IVF_STANDARD	1	75	0.099	0.066	229.8
EUCLIDEAN	KPP_ON	IVF_STANDARD	5	75	0.340	0.210	1134.1
EUCLIDEAN	KPP_ON	IVF_STANDARD	10	75	0.604	0.332	2274.4
EUCLIDEAN	KPP_ON	IVF_STANDARD	25	75	1.405	0.530	5687.0
EUCLIDEAN	KPP_ON	IVF_REFINED	1	75	4.129	0.792	16829.1
EUCLIDEAN	KPP_ON	IVF_REFINED	5	75	4.136	0.792	16829.1
EUCLIDEAN	KPP_ON	IVF_REFINED	10	75	4.135	0.792	16829.1
EUCLIDEAN	KPP_ON	IVF_REFINED	25	75	4.165	0.797	16949.3
COSINE	N/A	BRUTE	N/A	N/A	5.506	1.000	50000.0
COSINE	KPP_OFF	IVF_STANDARD	1	98	0.101	0.062	222.8
COSINE	KPP_OFF	IVF_STANDARD	5	98	0.374	0.213	1117.8
COSINE	KPP_OFF	IVF_STANDARD	10	98	0.661	0.337	2240.9
COSINE	KPP_OFF	IVF_STANDARD	25	98	1.574	0.544	5601.6
COSINE	KPP OFF	IVF REFINED	1	98	0.132	0.085	314.4

COSINE	KPP_OFF	IVF_REFINED	5	98	0.335	0.213	1117.8
COSINE	KPP_OFF	IVF_REFINED	10	98	0.595	0.337	2240.9
COSINE	KPP_OFF	IVF_REFINED	25	98	1.409	0.544	5601.6
COSINE	KPP_ON	IVF_STANDARD	1	98	0.102	0.075	224.0
COSINE	KPP_ON	IVF_STANDARD	5	98	0.344	0.229	1121.4
COSINE	KPP_ON	IVF_STANDARD	10	98	0.577	0.343	2247.1
COSINE	KPP_ON	IVF_STANDARD	25	98	1.361	0.563	5621.6
COSINE	KPP_ON	IVF_REFINED	1	98	0.107	0.098	288.9
COSINE	KPP_ON	IVF_REFINED	5	98	0.321	0.229	1121.4
COSINE	KPP_ON	IVF_REFINED	10	98	0.596	0.343	2247.1
COSINE	KPP_ON	IVF_REFINED	25	98	1.400	0.563	5621.6
=====							
=====							

6.2 The Core Speed/Accuracy Trade-off

The performance metrics highlight the clear trade-off between search speed and recall accuracy inside approximate nearest neighbor (ANN) indexes.

- Brute Force Baseline: Guarantees absolute accuracy (1.000 Recall@10), but requires scanning all 50,000 vectors, resulting in high execution times (6.195ms for Euclidean, 5.506ms for Cosine).
- IVF Standard (nprobe=1): Achieves sub-millisecond time (0.094ms), executing roughly 65x faster than Euclidean Brute Force. However, because it only searches the single best-matching cluster, it processes only ~224 vectors, causing recall to drop very low to 0.067.
- The Impact of Tuning NPROBE: As the nprobe parameter scales from 1 to 25, it searches in wider regional clusters. At nprobe=25, the database scans 5,627.1 vectors. This increases recall to 0.537, while latency remains well below brute force at 1.393ms.

6.3 Standard vs. Refined Execution Pathways

Euclidean Mode Anomaly

In Euclidean mode, regardless of the nprobe value, it scans ~16,774 vectors (~33% of the database), keeping time fixed at ~4.1ms and recall flat at 0.794.

Cosine Mode Optimization

There is a slight increase in recall and scanned vectors for nprobe=1 but identical for all other nprobes.

6.4 K-Means++ Initialization Behavioral Dynamics

Euclidean Clustering Optimization

For Euclidean distance metrics, enabling K-Means++ delivers a massive optimization benefit. Better initial placement drops the required optimization loops from 83 down to 75 iterations, shortening overall index build time from 104.27s to 94.23s.

